Team System322

IUTian Threads

Lab 10- Process Specifications and Structured Decisions

CSE 4408: System Analysis & Design Lab
August 21, 2025

THE TEAM

Sadman Shaharier Mahim
220041133

Alfi Shahrin
220041153

Nayeemul Hasan Prince
220041125

Project Recap

- IUTian Threads is a university-specific, forum-style platform designed for students to share concerns, ask questions, and engage in structured discussions.

- Addresses the current lack of a centralized and effectively moderated issue discussion system at IUT and IUT-OSW.

Selected Primitive Processes

Process 2.4 **Posting & Anonymous Posting**	• Input: New post content, user credentials • Output: Stored post, moderation alert • Data Stores: In-Subforum Content Master
Process 2.1 **Store Comment**	• Input: Comment data, post reference • Output: Updated post with comment • Data Stores: In-Subforum Content Master
Process 2.2 **Register Vote**	• Input: Vote action, post/comment ID • Output: Updated vote count • Data Stores: In-Subforum Content Master

Process Specification Form - 1

Number **Name** **Description**	2.4 Posting & Anonymous Posting Handles creation of new posts including anonymous posts with content validation & moderation alerts
Input Data Flow • New post content from student, user authentication token	**Output Data Flow:** • Stored post to In-Subforum Content Master, moderation alert to moderator
Type of Process: ☑ Online ☐ Batch ☐ Manual	
Decision Analysis: ☐ Structured English ☑ Decision Table ☐ Decision Tree	

Decision Table - Posting & Anonymous Posting

Conditions & Actions	Rule 1	Rule 2	Rule 3	Rule 4
User is Authenticated?	Y	Y	Y	Y
Post type=Anonymous?	Y	N	Y	N
Content Passes Validation?	Y	Y	N	N
Actions				
Store post with hidden author ID	X			
Store post with visible author ID		X		
Reject post submission			X	X
Send Moderation alert	X		X	X

Logic & Justification

Process Logic

- The system validates user authentication and post type to determine storage method.
- Anonymous posts hide author identity while maintaining accountability.
- All successful submissions trigger moderation workflows.

Justification

- User authentication, post type, and content validation all interact to determine the appropriate action.
- Each combination of conditions maps to specific actions, avoiding ambiguity in post handling.
- Successful posts automatically trigger moderation alerts for content review.

Process Specification Form - 2

Number **Name** **Description**	2.1 Store Comment Processes and stores user comments (and replies) on posts with validation, threading, and moderation cues.

Input Data Flow	**Output Data Flow:**
• comment_content (text) • target_post_ID (ID of post being commented on) • parent_comment_ID (for threaded reply) • user_auth_token / user_UUID	• Updated *In_Subforum_Content_Master* (append comment_record, increment comment_count) • success_confirmation • moderation_alert to *Moderation_Queue*

Type of Process: ☑ Online ☐ Batch ☐ Manual

Decision Analysis: ☑ Structured English ☐ Decision Table ☐ Decision Tree

Structured English- Store Comment

```
BEGIN Store_Comment
    RECEIVE comment_content, post_id, user_auth
    IF user_auth valid THEN
        IF post_id EXISTS IN In_Subforum_Content_Master THEN
            VALIDATE comment_content
            IF valid THEN
                CREATE & APPEND comment_record
                UPDATE comment_count
                SEND success_confirmation
            ELSE SEND validation_error
        ELSE SEND post_not_found_error
    ELSE SEND authentication_error
END Store_Comment
```

Note: Underlined terms are Data Dictionary entities/stores/services: *In_Subforum_Content_Master, User_Master, Activity_Log, Moderation_Queue, Auth_Service, Policy_Filter, Rate_Limiter, User_Interface*.

Structured English - Store Comment (Contd.)

Logic

- The system receives comment_content, post_id, and user_auth.
- It checks authentication; if invalid, returns an error.
- If valid, it verifies whether the post exists; if not, returns a not-found error.
- If the post exists, it validates the comment content; if invalid, returns a validation error.
- If valid, it creates the comment record, appends it, updates the count, and sends confirmation.

Justification

- **Sequential Flow:** Logic follows a step-by-step validation order.
- **Nested Conditions:** Each check depends on the previous one (auth → post → validation).
- **Clear Responses:** Every failure point sends a meaningful error; successful path updates the database and confirms.
- **Best Fit:** Structured English is ideal as it represents these sequential, nested validations more clearly than a table or tree.

Process Specification Form - 3

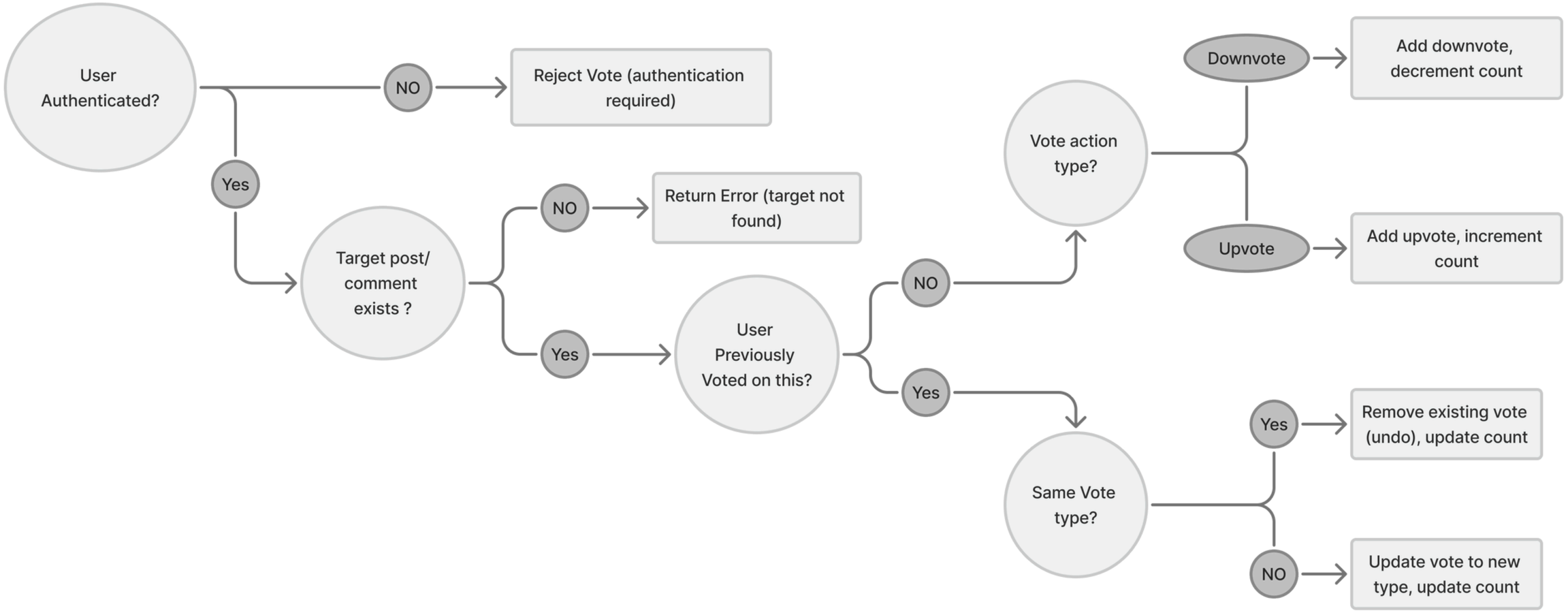
Number **Name** **Description**	2.2 Register Vote Processes upvotes and downvotes on posts/comments with duplicate vote prevention

Input Data Flow	**Output Data Flow:**
• Vote Action (upvote/downvote) • target_post_ID of the post interacted with • UUID of the user.	• Updated vote count in In-Subforum Content Master • Post update

Type of Process: ☑ Online ☐ Batch ☐ Manual

Decision Analysis: ☐ Structured English ☐ Decision Table ☑ Decision Tree

Process Specification Form - 3



Process Specification Form - 3

Logic

- Vote registration involves cascading decisions where each choice determines the next step.
- The tree structure naturally represents this branching logic and makes the duplicate vote handling clear.

Justification

- The vote processing involves multiple decision points, making a tree structure ideal.
- Each decision depends on the outcome of previous decisions, creating a natural tree-like flow.
- Each branch endpoint leads to a specific action, making the logic easy to follow.
- The tree clearly shows how the system handles duplicate votes and vote changes.

Ambiguities & Unresolved Issues

- **Anonymous Post Traceability**: Balancing user anonymity with moderation accountability - how to track anonymous posts for abuse prevention while maintaining privacy.
- **Content Validation Rules**: Defining comprehensive prohibited content filters without over-censoring legitimate academic discussions.
- **Vote Manipulation**: Preventing coordinated voting attacks or bot-driven vote manipulation in the university environment.
- **Concurrent Operations**: Handling simultaneous votes, comments, or posts on the same content without data conflicts.
- **Moderation Escalation**: Determining when content requires immediate moderator attention versus standard review queue processing.

Thank you!

We are open for Q&A